

Unit 5 – Reliable Group Communication & Naming Systems

These notes explain every topic and subtopic from the uploaded Unit 5 PDF in a simpler and more student-friendly way. Examples are included throughout so that the concepts are easier to understand and remember for exams.

Reliable Group Communication

Reliable multicast means sending one message to many processes in such a way that everyone correctly receives it.

Why is it important?

- In distributed systems, the same data may be copied (replicated) on many computers.
- If one message is lost, different systems may have different data.
- Reliable multicast helps keep all systems consistent.

Example:

Imagine a teacher sending exam marks to 100 students.

If some students receive the marks and others do not, confusion occurs.

Reliable multicast ensures all intended students receive the same message.

Main Ideas:

- Reliable point-to-point communication works between two systems.
- Group communication is harder because many receivers are involved.
- Systems must handle:
 - process joins
 - process leaves
 - crashes

Basic Techniques:

- Sequence numbers help detect missing messages.
- ACK (Acknowledgment) = "I received the message."
- NACK (Negative Acknowledgment) = "I missed the message."
- Sender stores messages temporarily in a history buffer for retransmission.

Scalability in Reliable Multicasting

Problem: Feedback Implosion

If 10,000 receivers send ACKs to the sender, the sender becomes overloaded.

Example:

A celebrity livestream asks viewers to confirm receipt of a notification.

Millions replying at once can crash the system.

Solution: NACK-based approach

- Receivers send feedback only if they missed a message.
- This reduces unnecessary responses.

Limitation:

If many users miss the same message, they may all send NACKs together.
This still creates overload.

Another Issue:

The sender must store old messages for a long time in case retransmission is needed.

Non-Hierarchical Feedback Control (SRM)

SRM = Scalable Reliable Multicast

Main Idea:

Instead of everyone sending ACKs, receivers send NACKs only when needed.

Feedback Suppression:

- Every receiver waits for a random amount of time before sending a NACK.
- If it hears another receiver already sent the same NACK, it cancels its own.

Example:

Four students miss a lecture PDF.

Student A asks first.

The others see the request and do not ask again.

Advantages:

- Reduces duplicate requests
- Reduces network traffic

Limitations:

- Requires proper timer management
- Some receivers still process unnecessary NACKs
- Complex at large scale

Hierarchical Feedback Control

Used for very large multicast groups.

Structure:

- Systems are divided into subgroups.
- Each subgroup has a coordinator.

- Coordinators form a tree structure.

Working:

- Local coordinator manages retransmissions.
- Coordinators communicate upward and downward.

Example:

Think of a school system:

- Principal → Teachers → Students

The principal does not communicate with every student directly.

Advantages:

- Better scalability
- Reduced load on sender

Disadvantages:

- Maintaining the hierarchy is difficult
- Dynamic updates are complex

Atomic Multicast

Atomic multicast guarantees:

1. All-or-nothing delivery
2. Same order delivery

This is important in replicated databases.

Example:

Suppose a bank updates account balances on multiple servers.

If one server misses an update, balances become inconsistent.

Atomic multicast ensures:

- Either all servers apply the transaction
- Or none apply it

Benefits:

- Consistency
- Correct synchronization
- Safe recovery after crashes

Virtual Synchrony

Virtual synchrony ensures all processes see the same sequence of events.

Basic Flow:

1. Message arrives from network
2. Communication layer buffers it
3. Message delivered to application

Key Property:

If a sender crashes while sending:

- Either everyone receives the message
- Or nobody receives it

Example:

A process sends a message and crashes midway.

Some systems may receive it while others may not.

Virtual synchrony discards partial delivery to maintain consistency.

Advantages:

- Consistent state
- Easier distributed programming
- Better fault handling

Message Ordering

Different multicast systems provide different ordering guarantees.

1. Unordered Multicast

- Messages are reliably delivered
- Order may differ at different receivers

Example:

P2 receives m1 then m2.

P3 receives m2 then m1.

2. FIFO Ordered Multicast

FIFO = First In First Out

Rule:

Messages from the same sender must be delivered in sending order.

Example:

If P1 sends:

m1 then m2

All receivers must receive:

m1 before m2

Different senders can still be interleaved differently.

3. Causally Ordered Multicast

If one message influences another, the order must be preserved.

Example:

P1 sends "Exam postponed."

P2 reads it and sends "Great news!"

Everyone must see:

"Exam postponed" before "Great news!"

Vector timestamps are commonly used.

4. Totally Ordered Multicast

All systems deliver all messages in exactly the same order.

Example:

If one process receives:

$m1 \rightarrow m3$

Then all processes must also receive:

$m1 \rightarrow m3$

Combining total ordering with virtual synchrony gives Atomic Multicast.

Naming in Distributed Systems

Names identify entities like users, files, services, or systems.

Main Uses:

1. Human-Friendly Names

Easy for humans to remember.

Example:

www.google.com

2. Location-Independent Naming

Names should work even if the entity moves.

Example:

Mobile phones continue working while moving between towers.

3. Attribute-Based Naming

Search entities using properties.

Example:

Find all printers:

- color = yes
- location = lab

Address

A name identifies an entity.

An address identifies where the entity can be reached.

Example:

- Person = Entity
- Phone = Access point
- Phone number = Address

In distributed systems:

IP address + port number = Address

Why addresses are not enough:

- They can change
- Systems may move
- Services may exist on multiple servers

Identifiers

Identifiers uniquely identify entities.

Properties:

1. Unique
2. No ambiguity
3. Persistent (not reused)

Example:

Student roll number is a better identifier than a nickname.

Name Resolution

Name resolution converts names into addresses.

Example:

www.google.com → IP address

Process:

- User provides name
- Naming system finds corresponding address

Large systems use hierarchical lookup.

Example Path:

root → nl → vu.nl → cs.vu.nl

Flat Naming

Flat names are random identifiers with no location information.

Problem:

How do we find the entity if the name gives no clue?

Solutions:

- Broadcasting
- Multicasting

Broadcasting

A request is sent to all machines.

Example:

“Who owns this IP address?”

ARP (Address Resolution Protocol) uses broadcasting.

Advantages:

- Simple

Disadvantages:

- Wastes bandwidth
- Not scalable

Multicasting

Instead of sending to everyone, send only to a group.

Example:

A mobile computer joins a multicast group.

When asked “Where is A?” it replies with its current location.

Uses:

1. Entity location
2. Locating replicated entities

Advantages:

- More efficient than broadcasting

Forwarding Pointers

When an entity moves:

- Old location stores a pointer to new location.

Example:

You move houses and leave a forwarding address at the old house.

Advantages:

- Easy to implement

Disadvantages:

- Long chains
- Slow lookup
- Broken links possible

Optimization:

Clients create shortcuts after learning the latest location.

Home-Based Approaches

Each entity has a home location.

The home:

- Tracks current location
- Forwards requests

Example:

Mobile IP

Advantages:

- Avoids long forwarding chains
- More scalable

Disadvantages:

- Home becomes bottleneck
- Single point of failure

Hierarchical Approaches

Network is divided into domains and subdomains.

Structure:

Root → domains → subdomains → leaf domains

Each domain has a directory node.

Lookup:

- Search moves upward until location found
- Then moves downward to actual entity

Advantages:

- Scalable
- Supports mobility and replication
- Exploits locality

Structured Naming

Structured names are human-readable.

Example:

/home/student/notes.txt

Namespace:

A graph structure containing:

1. Leaf nodes
2. Directory nodes

Leaf nodes:

Store actual data.

Directory nodes:

Store references to other nodes.

Name Resolution in Structured Naming

Path names are resolved step-by-step.

Example:

/home/student/file.txt

The system:

- Starts from root
- Traverses directories
- Finds final file

Closure Mechanism:

Determines where resolution starts.

Linking and Mounting

Aliases allow multiple names for the same entity.

Hard Link:

Two path names refer to same file.

Example:

/keys and /home/steen/keys

Symbolic Link:

Stores another path name.

Example:

Shortcut in Windows.

Mounting:

Combines multiple name spaces.

Example:

NFS allows accessing remote directories as if local.

Implementation of Name Space

Large systems divide name spaces across many servers.

Layers:

1. Global Layer

- Stable
- Highly available

2. Administrative Layer

- Managed by organizations

- Moderate changes

3. Managerial Layer

- Frequently changing
- Local data

Example:

DNS uses zones handled by different servers.

Trade-Off:

Replication improves availability but creates consistency issues.

Implementation of Name Resolution

Local resolver communicates with name servers.

Two Methods:

1. Iterative Resolution

Client contacts each server one by one.

Advantages:

- Lower server load

Disadvantages:

- More communication cost

2. Recursive Resolution

Servers contact other servers on behalf of client.

Advantages:

- Faster repeated queries
- Better caching

Disadvantages:

- Higher server load

Case Study: DNS

DNS = Domain Name System

Purpose:

Converts domain names into IP addresses.

Example:
flits.cs.vu.nl.

DNS Structure:

- Hierarchical tree
- Root represented by a dot

Domains:

Subtrees of DNS hierarchy.

Resource Records:

DNS stores information using records such as:

- A
- MX
- NS
- PTR
- TXT

Implementation:

- Zones handled by name servers
- Primary server handles updates
- Secondary servers copy data for availability

Attribute-Based Naming

Entities are searched using attributes.

Example:

Find:

- printer
- color = yes
- floor = 2

This system returns matching entities.

Directory Services:

Attribute-based naming systems are called directory services.

Example:

LDAP (Lightweight Directory Access Protocol)

RDF Model:

Resources represented as:

(subject, predicate, object)

Example:

(person, name, Alice)

Important Exam Tip: Focus on understanding the examples and flow of each concept instead of memorizing definitions word-for-word. Most distributed systems questions become easier when you visualize real-life communication and networking examples.